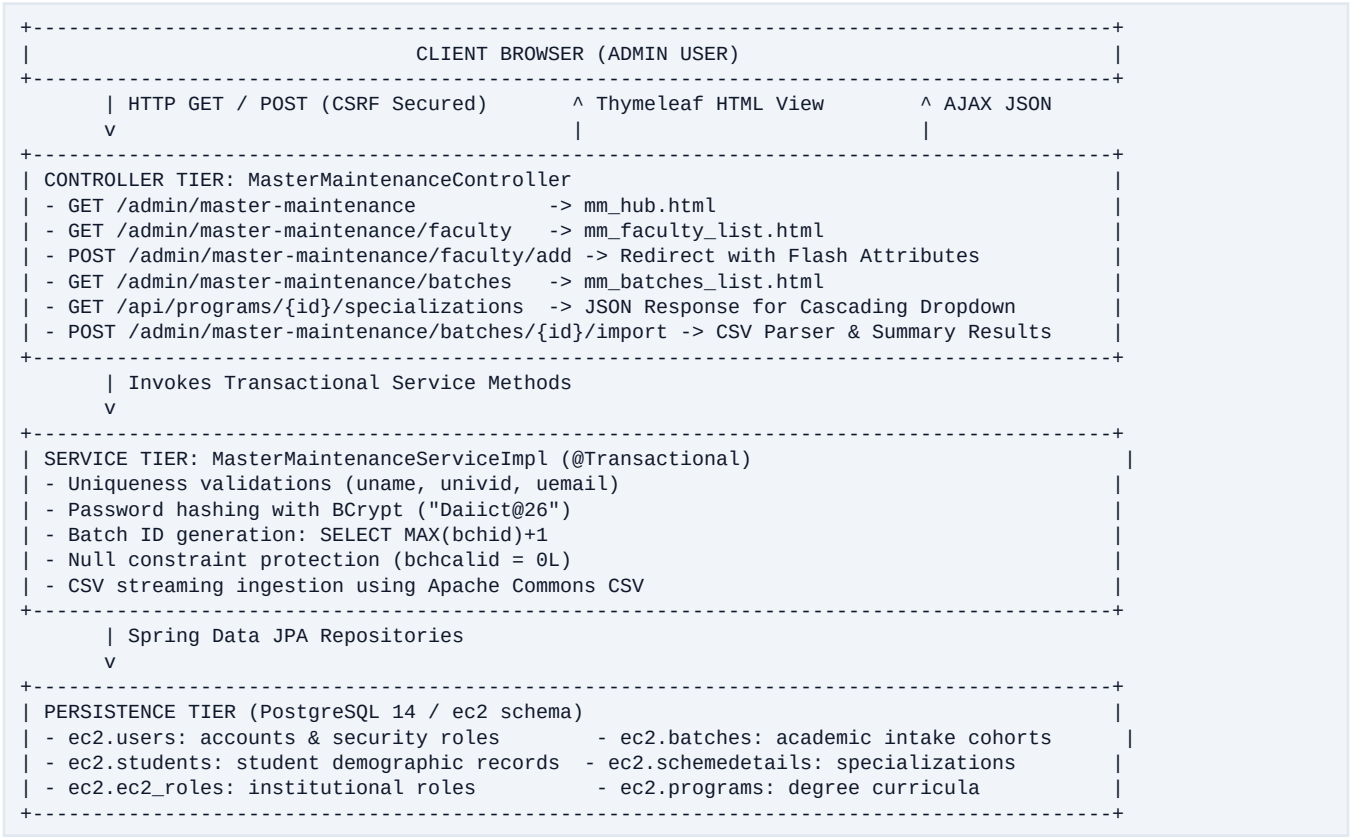
Master Maintenance Architecture & Flow

Comprehensive Sequence Diagrams, Code Analysis & Defense Guide

System: DAU Academic Portal	**Framework:** Spring Boot 3.2.1 / Java 17
Database: PostgreSQL (ec2 schema)	**Security:** Spring Security 6 (BCrypt / Role-Based)
Scope: Faculty, Batches, CSV Import, Users, Roles	**Audience:** Academic Mentors & Software Reviewers

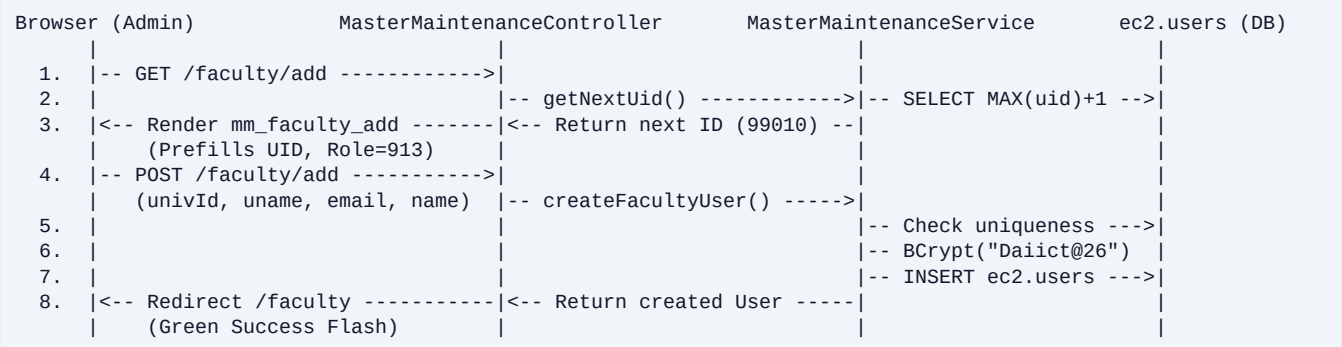
1. System Architecture & Tier Interaction

The Master Maintenance feature implements a clean multi-tier Spring MVC architecture. Incoming requests pass through Spring Security's authorization filter, are dispatched by MasterMaintenanceController, execute transaction-safe business rules in MasterMaintenanceService, and persist into PostgreSQL via Spring Data JPA.



2. Module 1: Add New Faculty User

Execution Sequence Diagram:



Service Code Implementation:

```
@Override
@Transactional
public Users createFacultyUser(UserCreateForm form, Long adminId) {
    form.setUrole(UserRoles.FACULTY); // Role code locked to "913"
    return saveNewUserInternal(form, adminId, true);
}

private Users saveNewUserInternal(UserCreateForm form, Long adminId, boolean isFaculty) {
    // 1. Natural Key Uniqueness Validations
    if (userRepository.existsByUnivIdIgnoreCase(form.getUnivId().trim())) {
        throw new IllegalArgumentException("University ID '" + form.getUnivId() + "' already exists.");
    }
    if (userRepository.existsByUnameIgnoreCase(form.getUname().trim())) {
        throw new IllegalArgumentException("Username '" + form.getUname() + "' already exists.");
    }
    if (userRepository.existsByUemailIgnoreCase(form.getUemail().trim())) {
        throw new IllegalArgumentException("Email '" + form.getUemail() + "' already exists.");
    }

    // 2. Safe Auto-Increment ID Resolution
    Long uid = (form.getUid() != null && form.getUid() > 0) ? form.getUid() : userRepository.findNextUid();
    if (userRepository.existsByUid(uid)) {
        uid = userRepository.findNextUid();
    }

    // 3. User Entity Construction
    Users user = new Users();
    user.setUid(uid);
    user.setUnivId(form.getUnivId().trim());
    user.setUname(form.getUname().trim());
    user.setUfullname(form.getUfullname().trim());
    user.setUrole(form.getUrole().trim());
    user.setUemail(form.getUemail().trim());
    user.setPassword(passwordEncoder.encode("Daiict@26")); // BCrypt encrypted
    user.setStatus(true);
    user.setRowState((short) 1);
    user.setCreatedBy(adminId != null ? adminId : 0L);
    user.setCreateTs(LocalDateTime.now());

    return userRepository.save(user);
}
```

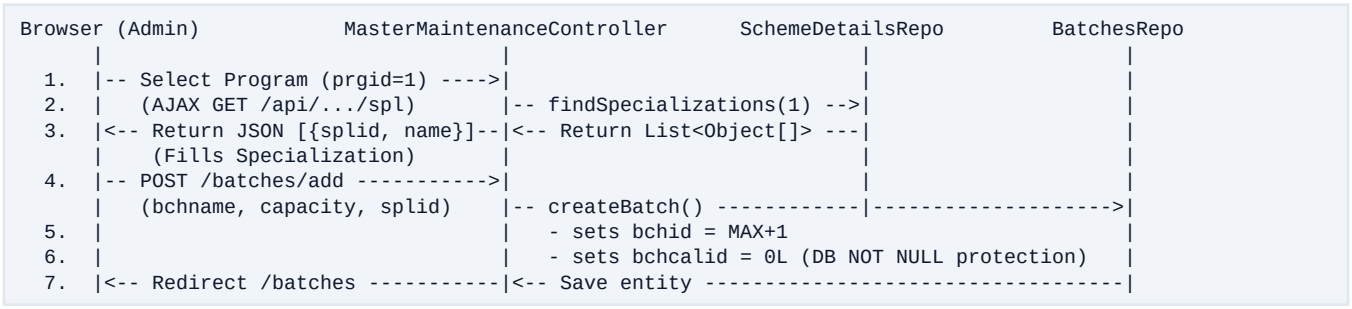
Code Explanation & Key Decisions:

- **Dual Natural Key Uniqueness:** Validates both univId (Employee ID) and uname (Login ID) independently to avoid login collisions.
- **Security Compliance:** Uses BCryptPasswordEncoder for initial password Daiict@26. Plaintext is never stored in DB.

- **Role Code Locking:** Hardcodes urole = 913; faculty cannot accidentally be given elevated privileges.

3. Module 2: Batches & Bulk CSV Student Import

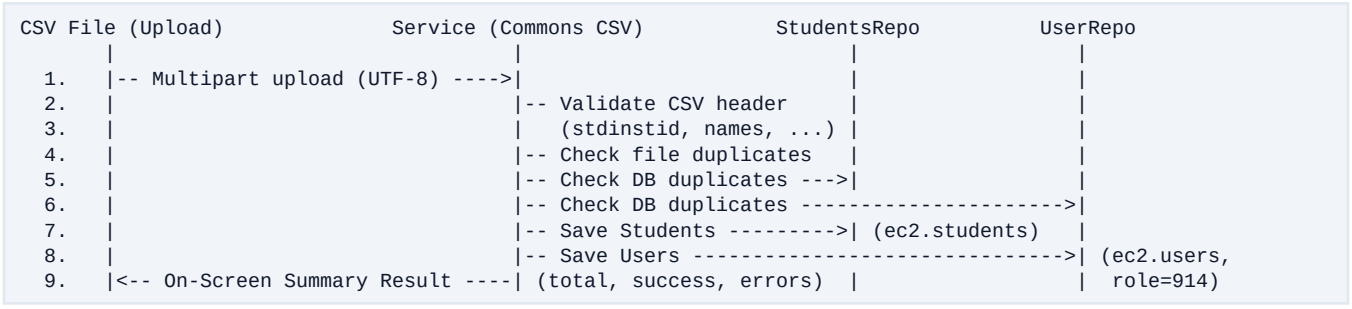
Sequence Diagram 1: Cascading Specialization & Batch Creation



Batch Creation Logic:

```
// MasterMaintenanceServiceImpl.java: createBatch()
@Override
@Transactional
public Batches createBatch(BatchCreateForm form, Long adminId) {
    Programs program = programsRepository.findById(form.getBchprgid())
            .orElseThrow(() -> new IllegalArgumentException("Selected program does not exist."));

    if (batchesRepository.existsByBchprgidAndBchname(form.getBchprgid(), form.getBchname().trim())) {
        throw new IllegalArgumentException("Batch '" + form.getBchname().trim() + "' already exists for program.");
    }

    Long nextId = form.getBchid() != null && form.getBchid() > 0 ? form.getBchid() : batchesRepository.findNextBatchId();

    Batches batch = new Batches();
    batch.setBchid(nextId);
    batch.setPrograms(program);
    batch.setBchprgid(form.getBchprgid());
    batch.setBchname(form.getBchname().trim());
    batch.setBchcapacity(form.getBchcapacity());
    batch.setBchinstcode(form.getBchinstcode() != null ? form.getBchinstcode().trim() : "DAU");
    batch.setSchemeId(form.getSchemeId());
    batch.setSplid(form.getSplid());
    batch.setBchcalid(0L); // Critical: satisfys PostgreSQL NOT NULL constraint while skipping calendar
    batch.setBchrowstate(1L);
    batch.setBchcreatedby(adminId != null ? adminId : 0L);
    batch.setBchcreatedat(LocalDateTime.now());
    batch.setBchlastupdatedby(adminId != null ? adminId : 0L);
    batch.setBchlastupdatedat(LocalDateTime.now());

    return batchesRepository.save(batch);
}
```

Sequence Diagram 2: Bulk CSV Student & User Ingestion



CSV Parsing & Dual Entity Ingestion Code:

```
// MasterMaintenanceServiceImpl.java: importStudentsFromCsv() (Core ingestion loop)
for (CSVRecord record : parser) {
    String instId = getCol(record, "stdinstid", "studentid", "instid");
    String firstName = getCol(record, "stdfirstname", "firstname");
    String lastName = getCol(record, "stdlastname", "lastname");
    String email = getCol(record, "stdemail", "email");

    // Check 1: In-File Duplicate Rejection (Tracking sets)
    if (instId == null || !seenInstIds.add(instId.toLowerCase())) {
        result.addError("Duplicate or missing Student ID: " + instId);
        result.setErrorCount(result.getErrorCount() + 1);
        continue;
    }
    // Check 2: Database Conflict Rejection
    if (studentsRepository.existsByStdinstidIgnoreCase(instId)) {
        result.addError("Student ID '" + instId + "' already exists in database.");
        result.setErrorCount(result.getErrorCount() + 1);
        continue;
    }

    // Step 1: Persist Student Record into ec2.students
    Students student = new Students();
    student.setStdid(nextStdId);
    student.setStdbchid(batch.getBchid());
    student.setBatch(batch);
    student.setStdinstid(instId);
    student.setStdfirstname(firstName);
    student.setStdlastname(lastName);
    student.setStdemail(email);
    student.setStdstatus("A");
    student.setStdrowstate(1L);
    studentsRepository.save(student);

    // Step 2: Persist Student User Account into ec2.users
    Users user = new Users();
    user.setUid(nextUid);
    user.setStdid(nextStdId);
    user.setUnivId(instId);
    user.setUname(instId);
    user.setUfullname(firstName + " " + lastName);
    user.setUrole(UserRoles.STUDENT); // Role "914"
    user.setPassword(passwordEncoder.encode("Daiict@26"));
    user.setStatus(true);
    user.setRowState((short) 1);
    userRepository.save(user);

    nextStdId++; nextUid++;
    result.setSuccessCount(result.getSuccessCount() + 1);
}
```

Key Points for Mentor Defense:

- **Database Safety:** Column bchcalid has a NOT NULL constraint in PostgreSQL; setting default 0L satisfies schema rules without requiring calendar configuration.
- **Atomic Dual Insertion:** Both student demographic info and student login credentials are created in sync. Students log in immediately with their Institute ID.
- **Resilient Error Isolation:** Invalid rows (e.g. malformed date or duplicate ID) are skipped and reported in the on-screen summary without stopping the remaining valid rows.

4. Modules 3 & 4: User Management & Role Master

Module 3: Add User (Any Non-Student Role)

Allows creating administrative, dean, registrar, and coordinator accounts. The role dropdown dynamically loads from ec2.ec2_roles, displaying institutional role names while saving rid. **Student role (914) is strictly excluded**.

```
@Override
public List<Role> getAvailableRolesForUser() {
    // Queries ec2_roles WHERE rid != '914' ORDER BY rolename ASC
    return roleRepository.findByRidNotOrderByRoleNameAsc(UserRoles.STUDENT);
}

@Override
@Transactional
public Users createUser(UserCreateForm form, Long adminId) {
    if (UserRoles.STUDENT.equals(form.getUrole().trim())) {
        throw new IllegalArgumentException("Student users cannot be created directly here. Use Batch Import instead.");
    }
    if (!roleRepository.existsByRid(form.getUrole().trim())) {
        throw new IllegalArgumentException("Invalid role selected: " + form.getUrole());
    }
    return saveNewUserInternal(form, adminId, false);
}
```

Module 4: Security Role Master

Provides dynamic configuration of institutional roles. Validates that both rid (Role Code) and roleName are strictly unique before inserting into ec2.ec2_roles.

```
@Override
@Transactional
public Role createRole(RoleCreateForm form, Long adminId) {
    String rid = form.getRid().trim();
    String roleName = form.getRoleName().trim();

    if (roleRepository.existsByRid(rid)) {
        throw new IllegalArgumentException("Role ID '" + rid + "' already exists.");
    }
    if (roleRepository.existsByRoleNameIgnoreCase(roleName)) {
        throw new IllegalArgumentException("Role Name '" + roleName + "' already exists.");
    }

    Role role = new Role();
    role.setRid(rid);
    role.setRoleName(roleName);
    role.setComments(form.getComments() != null ? form.getComments().trim() : null);
    role.setCreatedBy(adminId != null ? adminId : 0L);
    role.setCreateTs(LocalDateTime.now());
    role.setRowState((short) 1);

    return roleRepository.save(role);
}
```

5. Debugging & Troubleshooting Matrix for Mentors

Scenario / Problem	Diagnostic Technique & Code Resolution
Duplicate Key Exception (users_pkey / batches_pkey)	Occurs if manual ID conflicts with DB. Resolution: findNextUid() and findNextBatchId() execute COALESCE(MAX()+1) dynamically before persist.
bchcalid NOT NULL Violation	Column has NOT NULL constraint in ec2.batches schema. Resolution: Default fallback 0L is set when skipping academic calendar assignment.
Right Viewport Dropdown Overflow	Fixed by applying dropdown-menu-end and CSS rule right: 0 !important; left: auto !important; to .nav-right .dropdown-menu.
CSV row skipped with Error Log	Duplicate student ID or email in CSV or DB, or malformed date. Review on-screen error summary for row number and exact rejection message.